

Operator Decision Brief — Security Response Headers (Item O) + DDoS Default Posture (Item Q)

Revision: 1 **Last modified:** 2026-07-09T20:14:51Z **Status:** OPERATOR-GATED PROPOSAL — research + draft only, NOTHING wired into the tree **Authority:** research/decision brief per §11.4.101 (autonomous-decision-over-blocking) + §11.4.122 (no silent change to shipped behaviour). This document proposes; the operator decides. **Scope:** server/ (Gin control plane, single-binary API + embedded OTA-Manager SPA)

0. Honest boundary (§11.4.6)

This is a **proposal backed by research**, NOT an implementation. No middleware was added, no `server.go` line changed, no default flipped. The code in §3.4 and §4.4 is a **draft** for the operator to read and approve — it is deliberately un-applied. Every current-state claim below is cited to `file:line` in the working tree at the time of writing. The two decisions in §5 are the operator's to make; each option's risk is stated so the choice is informed, not guessed.

1. Item O — Security response headers

1.1 CURRENT STATE (established as FACT)

No security response headers are set anywhere in the server. A tree-wide grep for Strict-Transport-Security, X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Referrer-Policy, Permissions-Policy, Cross-Origin-*, X-XSS-Protection across server/**/*.go (excluding tests) returns **zero matches**.

The complete global middleware chain is:

```
server/internal/api/server.go:117
    r.Use(recoveryMiddleware(), requestIDMiddleware(),
        maxInflightMiddleware(s.cfg.MaxInflight),
compressionMiddleware())
```

The only response headers set today are:

Header	Where	Value
X-Request-Id	middleware.go:29 (requestIDMiddleware)	random 16-byte hex correlation id
Vary: Accept-Encoding	compression.go:24 (compressionMiddleware; supersedes the unused varyMiddleware at middleware.go:37)	Accept-Encoding
Retry-After: 1	rate_limit.go:28	only on a 429 shed
Content-Type	per-handler	application/json / text/html / text/ plain

Headers that ARE set: X-Request-Id, Vary, Content-Type, and Retry-After (on 429 only).

Headers that are NOT set (absent): Strict-Transport-Security (HSTS), X-Content-Type-Options, X-Frame-Options, Content-Security-Policy, Referrer-Policy, Permissions-Policy, Cross-Origin-Opener-Policy, Cross-Origin-Resource-Policy, Cross-Origin-Embedder-Policy, X-Permitted-Cross-Domain-Policies, X-DNS-Prefetch-Control, Cache-Control.

Serving topology relevant to CSP (FACT): the server is a single Go binary that serves BOTH the JSON API (default base path `/api/v1`) AND the built OTA-Manager SPA embedded via `//go:embed manager-dist/*` and mounted at `/manager` (`embed.go:51,68`). The SPA is a Vite/React production build (`embed.go:8-9`) served same-origin as the API, so `embed.go:36-37` notes “No CORS is needed ... same origin”. This is the key constraint for CSP: the manager UI’s own JS/CSS/font assets live under `/manager/assets/` and are same-origin, so a ‘self’-based CSP is compatible — see §1.3 caveats.

TLS is **optional**: HTTP/3+HTTP/2 over TLS is enabled only when both `HELIX_TLS_CERT` and `HELIX_TLS_KEY` are set, otherwise the binary serves plain HTTP on `HELIX_PORT` (`config.go:79-84`). This matters for HSTS (§1.3).

1.2 RESEARCH — OWASP Secure Headers best practice

Source: **OWASP Secure Headers Project**, machine-readable “headers to add” and “headers to remove” recipes, `last_update_utc: 2026-06-30` (verified 2026-07-09, well inside the \$11.4.99 90-day freshness window). Recommended add-set:

Header	OWASP recommended value
Strict-Transport-Security	<code>max-age=63072000;</code> <code>includeSubDomains</code>
X-Content-Type-Options	<code>nosniff</code>
X-Frame-Options	<code>deny</code>
Content-Security-Policy	<code>default-src 'self'; form-action</code> <code>'self'; base-uri 'self';</code> <code>object-src 'none'; frame-</code> <code>ancestors 'none'; upgrade-</code> <code>insecure-requests</code>
Referrer-Policy	<code>no-referrer</code>
Permissions-Policy	(long deny-list: <code>accelerometer=()</code> , <code>autoplay=()</code> , <code>camera=()</code> , <code>geolocation=()</code> , <code>microphone=()</code> , <code>payment=()</code> , <code>usb=()</code> , ... — every feature denied)
Cross-Origin-Opener-Policy	<code>same-origin</code>
Cross-Origin-Resource-Policy	<code>same-origin</code>

Header	OWASP recommended value
Cross-Origin-Embedder-Policy	require-corp
Cache-Control	no-store, max-age=0
X-Permitted-Cross-Domain-Policies	none
X-DNS-Prefetch-Control	off
Clear-Site-Data	"cache", "cookies", "storage" (logout responses only)

OWASP “headers to remove” (fingerprinting reduction): Server, X-Powered-By, X-AspNet-Version, X-Generator, and many framework/proxy trailers. Gin does **not** emit a Server header by default, so there is little to strip here — but if the deployment sits behind Nginx/Envoy, those hops should suppress their own Server/X-Powered-By (out of scope for this Go change).

1.3 HONEST TRADEOFFS — headers that could BREAK the embedded SPA

These are the load-bearing caveats. Applying the raw OWASP set globally to a single-binary API+SPA **will break the manager UI or the API**. Per-header:

1. **Cache-Control: no-store, max-age=0 applied globally → BREAKS asset caching.** OWASP’s no-store is correct for API/auth JSON (never cache a token or a device list), but if applied to the SPA’s hashed static assets under /manager/assets/* it defeats the entire point of Vite content-hashed filenames (the browser re-downloads the bundle every navigation). **Resolution:** scope no-store to API + text/html responses; let hashed assets keep the default (or Cache-Control: public, max-age=31536000, immutable). This is why the draft applies no-store in the API-group middleware, not the SPA path.
2. **Cross-Origin-Embedder-Policy: require-corp → HIGH breakage risk.** COEP require-corp demands that *every* subresource carry a CORP/CORS header or the browser blocks it. A Vite SPA that ever loads a font, image, or map from a CDN — or even a same-origin asset the static handler doesn’t tag — silently breaks. COEP’s only real benefit is unlocking SharedArrayBuffer/high-res timers, which the manager UI does not use. **Recommendation: DO NOT enable COEP** (omit it) unless a future feature needs cross-origin isolation.

3. **Content-Security-Policy — default-src 'self' is compatible, with two watch-items.** The Vite production build serves external `<script type="module" src="/manager/assets/...">` and `<link rel="stylesheet">`, all same-origin → allowed by 'self'. Watch-items: (a) if the build emits any **inline** `<script>` or `inline style="" / <style>` without a hash/nonce, default-src 'self' blocks it — Vite React builds normally externalise JS but MAY inline tiny CSS or a runtime snippet; (b) style-src 'self' can block styled-component / injected inline styles → many SPAs need style-src 'self' 'unsafe-inline'. **Resolution:** ship a slightly relaxed CSP for the HTML document (style-src 'self' 'unsafe-inline', img-src 'self' data:, connect-src 'self') and validate against the real built SPA before enabling (see §5 decision + §6 validation plan). The strict API CSP (default-src 'none'; frame-ancestors 'none') is safe for JSON.
4. **Strict-Transport-Security on a plaintext dev listener → must be gated.** HSTS is meaningless (and per-spec ignored by browsers) over HTTP, but emitting it in dev is sloppy and, if a dev ever hits the box over an https proxy, can pin a bad policy. **Resolution:** emit HSTS **only when TLS is configured** (cfg.TLSCertFile != "" && cfg.TLSKeyFile != "", per config.go:83-84).
5. **X-Frame-Options: deny / frame-ancestors 'none' → intended, low risk.** The manager UI is not designed to be embedded in an iframe, so clickjacking protection is free. Only enable framing if a future dashboard embeds /manager.
6. **Referrer-Policy: no-referrer** is safe for both API and SPA (the manager UI makes only same-origin calls). strict-origin-when-cross-origin is a softer alternative if any outbound link telemetry is ever wanted; no-referrer is the stricter, recommended default here.

1.4 CONCRETE PROPOSAL — recommended header set (tailored, not raw-OWASP)

Two tiers, because the API and the SPA have different needs:

Tier A — applied to ALL responses (global securityHeadersMiddleware):

Header	Proposed value	Justification	SPA-compat caveat
	nosniff		

Header	Proposed value	Justification	SPA-compat caveat
X-Content-Type-Options		stops MIME-sniffing of API JSON + assets	none — always safe
X-Frame-Options	DENY	anti-clickjacking (UI not meant to be framed)	none unless future embed
Referrer-Policy	no-referrer	no referrer leakage of internal URLs/tokens	none (same-origin only)
Cross-Origin-Opener-Policy	same-origin	process-isolates the manager tab	none
Cross-Origin-Resource-Policy	same-origin	blocks cross-origin embedding of responses	none (single origin)
Permissions-Policy	deny-all sensitive features (geolocation=(), camera=(), microphone=(), payment=(), usb=(), ...)	denies powerful APIs UI never uses	none
X-Permitted-Cross-Domain-Policies	none	blocks Flash/PDF cross-domain policy abuse	none
Strict-Transport-Security	max-age=63072000; includeSubDomains — only when TLS configured	force HTTPS for 2y	gated to TLS mode (§1.3.4)

Tier B — API-only (JSON responses under the API base path):

Header	Proposed value	Justification
Content-Security-Policy	default-src 'none'; frame-ancestors	

Header	Proposed value	Justification
	'none'; base-uri 'none'	JSON needs no resources at all — maximally strict
Cache-Control	no-store	never cache tokens / device lists / audit logs

Tier C — SPA HTML document only (the /manager index response):

Header	Proposed value	Justification	Caveat
Content-Security-Policy	default-src 'self'; script-src 'self'; style-src 'self' 'unsafe-inline'; img-src 'self' data:; font-src 'self'; connect-src 'self'; object-src 'none'; base-uri 'self'; form-action 'self'; frame-ancestors 'none'; upgrade-insecure-requests	locks the UI to same-origin resources	'unsafe-inline' for style only; MUST be validated against the real built SPA (§6) before shipping — tighten to a nonce later

Deliberately OMITTED (with reason): - Cross-Origin-Embedder-Policy: require-corp — high breakage risk, no benefit (§1.3.2). - Global Cache-Control: no-store — would defeat hashed-asset caching (§1.3.1); scoped to Tier B instead. - X-XSS-Protection — deprecated/removed from OWASP; modern browsers ignore it; a stale 1; mode=block can introduce its own XSS vector. Do not add.

1.5 DRAFT middleware code (NOT applied — for operator review)

```
// server/internal/api/security_headers.go — DRAFT, un-wired
// proposal (Item 0)
package api

import "github.com/gin-gonic/gin"

// permissionsPolicyDenyAll denies every powerful browser feature
// the control
// plane never uses (OWASP Secure Headers, 2026-06-30 recipe,
// trimmed).
const permissionsPolicyDenyAll = "accelerometer=(), autoplay=(),
    camera=(), " +
    "cross-origin-isolated=(), display-capture=(), encrypted-
    media=(), " +

    "fullscreen=(), geolocation=(), gyroscope=(), keyboard-
    map=(), " +
    "magnetometer=(), microphone=(), midi=(), payment=(), picture-
    in-picture=(), " +
    "publickey-credentials-get=(), screen-wake-lock=(), sync-
    xhr=(self), usb=(), " +
    "web-share=(), xr-spatial-tracking=(), clipboard-read=(),
    clipboard-write=(), " +
    "hid=(), idle-detection=(), serial=()"

// securityHeadersMiddleware sets the Tier-A headers on EVERY
// response. HSTS is
// emitted only when TLS is configured (meaningless over plaintext
// HTTP, §1.3.4).
// It runs early in the chain (before handlers) so the headers are
// present even
// on error/shed responses.
func securityHeadersMiddleware(tlsEnabled bool) gin.HandlerFunc {
    return func(c *gin.Context) {
        h := c.Writer.Header()
        h.Set("X-Content-Type-Options", "nosniff")
        h.Set("X-Frame-Options", "DENY")
        h.Set("Referrer-Policy", "no-referrer")
        h.Set("Cross-Origin-Opener-Policy", "same-origin")
        h.Set("Cross-Origin-Resource-Policy", "same-origin")
        h.Set("Permissions-Policy", permissionsPolicyDenyAll)
        h.Set("X-Permitted-Cross-Domain-Policies", "none")
        if tlsEnabled {
            h.Set("Strict-Transport-Security", "max-age=63072000;
            includeSubDomains")
        }
    }
}
```



```

        c.Next()
    }
}

// apiSecurityHeadersMiddleware adds the Tier-B API-only headers.
// Applied to the
// v1 group (JSON), NOT to the /manager SPA path.
func apiSecurityHeadersMiddleware() gin.HandlerFunc {
    return func(c *gin.Context) {
        h := c.Writer.Header()
        h.Set("Content-Security-Policy", "default-src 'none';
        frame-ancestors 'none'; base-uri 'none'")
        h.Set("Cache-Control", "no-store")
        c.Next()
    }
}

// spaCSP is the Tier-C document CSP for the embedded manager UI.
// It is set on the
// index.html response in MountManagerUI (embed.go), NOT globally,
// so hashed
// assets keep their cacheable headers. MUST be validated against
// the real built
// SPA before enabling (§6).
const spaCSP = "default-src 'self'; script-src 'self'; style-src
'self' 'unsafe-inline'; " +
    "img-src 'self' data:; font-src 'self'; connect-src 'self';
    object-src 'none'; " +
    "base-uri 'self'; form-action 'self'; frame-ancestors 'none';
    upgrade-insecure-requests"

```

Wiring (also draft — shows exactly what would change in `server.go` :117):

```

// DRAFT — proposed Router() change. tlsEnabled :=
    s.cfg.TLSCertFile != "" && s.cfg.TLSKeyFile != ""
r.Use(recoveryMiddleware(), requestIDMiddleware(),

    securityHeadersMiddleware(tlsEnabled),           // NEW
    (Tier A, global)
    maxInflightMiddleware(s.cfg.MaxInflight),
    compressionMiddleware())
// ...
v1 := r.Group(s.cfg.APIBasePath)
v1.Use(apiSecurityHeadersMiddleware())              // NEW (Tier
    B, API-only)
// ...and in MountManagerUI, on the index.html branch:
    c.Header("Content-Security-Policy", spaCSP)

```

2. Item Q — DDoS default posture (HELIX_MAX_INFLIGHT)

2.1 CURRENT STATE (established as FACT)

The in-flight limiter exists and is correct, but is **DISABLED BY DEFAULT**.

- Config field + default: config.go:60-62 + config.go:126 if c.MaxInflight, err = getInt64("HELIX_MAX_INFLIGHT", 0); err != nil { ... } — **the default is 0**.

- Semantics of 0: rate_limit.go:17-20 —

```
func maxInflightMiddleware(limit int64) gin.HandlerFunc {
    if limit <= 0 {
        return func(c *gin.Context) { c.Next() } // no-op
        passthrough
    }
    // ... semaphore of capacity `limit`; excess → 429 + Retry-
    After: 1
}
```

So **unset (0 or negative) = UNLIMITED in-flight requests (no load-shedding)**.

- When enabled (limit > 0), the middleware is a buffered-channel semaphore: the limit-th+1 concurrent request is shed **immediately** with 429 RATE_LIMITED + Retry-After: 1 (rate_limit.go:21-32) — non-blocking, no queue, O(1). Behaviour is proven by rate_limit_test.go (TestMaxInflightShedsUnderFlood, TestMaxInflightDisabledByDefault).
- It is wired in the global chain at server.go:117.

Summary of fact: the DoS protection is built, tested, and wired — but ships **off**. A production deployment that never sets HELIX_MAX_INFLIGHT has **no in-flight ceiling**: a flood piles unbounded concurrent goroutines + memory onto the host until OOM.

2.2 RESEARCH — what a concurrency limiter should protect, and default sizing

The limiter protects memory / goroutines / downstream saturation, NOT CPU. Measured token-crypto hot path (operator-supplied benchmark, `token_bench_test.go` is the harness): a **forged-token reject** ≈ 473 ns and an **accepted-token verify** ≈ 1697 ns (HMAC-SHA256 + base64url + JSON). At ~ 1.7 μ s/verify a single core absorbs on the order of ~ 590 k auth checks/sec — i.e. the crypto path is **not** the bottleneck; the server can absorb very high request *rates* on CPU alone. What a flood actually exhausts is: - **Goroutine + stack memory** (Gin spawns a goroutine per connection/request); - **The pgx/PostgreSQL connection pool** (the production `store.Repository` target, `config.go:88-91`) — an unbounded in-flight count queues behind a bounded pool and balloons latency + memory; - **In-memory store contention** (MVP repo) and upload buffers (`MaxUploadBytes`).

So the correct mental model (consistent with the `rate_limit.go:10-16` comment “rather than piling up unbounded work on the host”): the in-flight cap is a **memory/back-pressure guard**, and its value should be sized to *how many concurrent requests the host’s RAM + downstream pool can safely carry*, not to CPU throughput. Industry Go/Gin concurrency-limiter guidance (gin-limit and similar middlewares) uses a fixed semaphore of concurrent slots and sheds the overflow — exactly what `rate_limit.go` already does; published example limits cluster in the low hundreds (e.g. ~ 20 – 200), but every source stresses the value is resource-derived, not a universal constant.

2.3 CONCRETE PROPOSAL — recommended default posture

Recommendation: ship a conservative NON-ZERO default so “install and forget” is safe, while keeping it fully tunable. Proposed default: `HELIX_MAX_INFLIGHT = 256` (per server instance), reasoning:

- 256 concurrent in-flight requests is far above any realistic control-plane steady state (this is a fleet-management API — a handful of operators + periodic device check-ins, not a public high-fanout service), so **legitimate traffic is never shed** at 256.
- 256 is comfortably below the point where per-request goroutines/buffers threaten host RAM or overwhelm a default pgx pool, so a genuine flood is **shed at 429** before OOM — the exact protection the middleware was built for.

- It is a round, greppable, easily-tuned starting value; large deployments raise it via env, tiny/embedded hosts lower it. Because the limiter is O(1) and non-blocking, an over-generous default costs nothing until saturation.

The value should ultimately be **evidence-based** (§11.4.24 / §11.4.6): 256 is a safe starting default, and the shipped number should be confirmed against a real saturation/DDoS benchmark on the target host (the §CONTINUATION “security saturation / DDoS” work item is the natural place to calibrate it).

2.4 DRAFT change (NOT applied — one line)

```
// server/internal/config/config.go:126 - DRAFT: flip the default
// 0 -> 256.
// (0 still available as an explicit opt-OUT:
//   `HELIX_MAX_INFLIGHT=0` = unlimited.)
if c.MaxInflight, err = getInt64("HELIX_MAX_INFLIGHT", 256); err != nil {
    return Config{}, err
}
```

No code in `rate_limit.go` changes — only the default constant. Note the polarity inversion this creates: after the change, `HELIX_MAX_INFLIGHT=0` becomes the explicit **opt-out** (unlimited) rather than the implicit default. That must be documented in `.env.example` and the deployment guide so an operator who *wants* unlimited sets `0` deliberately.

3. DECISIONS THE OPERATOR MUST MAKE

Decision O — which security headers to enable

Choice	What it means	Risk
O-1 (recommended): enable Tier A + B now; enable Tier C (SPA CSP) after §6 validation	Global hardening + strict API CSP ship immediately; the SPA document CSP ships once validated against the real built manager UI	LOW. Tier A/B cannot break the SPA (they don't touch its asset loading). The only deferred item is the SPA CSP, gated behind a validation step.
O-2: enable everything		MEDIUM. A too-strict <code>script-src/style-src</code>

Choice	What it means	Risk
including SPA CSP immediately	Full OWASP-aligned set in one change	can white-screen the manager UI if the Vite build inlines anything; requires the \$6 validation to run <i>before</i> merge, not after.
O-3: Tier A only (no CSP at all)	Cheapest, zero SPA risk	LOW risk / LOWER benefit — leaves the strongest XSS/data-exfil control (CSP) off.
O-4: do nothing	Ship as-is	HIGH (security). No clickjacking/MIME/HSTS/CSP protection on a control plane that mints bearer tokens.

Additional sub-decision if HSTS is enabled: **includeSubDomains** (proposed) and whether to add **preload** (NOT proposed by default — preload is a hard-to-reverse commitment that all current + future subdomains are HTTPS-only; only add with explicit intent).

Decision Q — default `HELIX_MAX_INFLIGHT`

Choice	Default value	Risk
Q-1 (recommended): ship 256	256 in-flight cap by default	LOW. Protects against flood-OOM out of the box; 256 is well above real steady-state so no false shedding. Introduces the <code>0=explicit-unlimited</code> polarity note (§2.4).
Q-2: keep 0 (unlimited) but document a recommended production value	0 (unchanged)	MEDIUM. Behaviour-preserving (no §11.4.122 concern), but a default install stays unprotected — relies on every

Choice	Default value	Risk
Q-3: a different explicit number (e.g. 128 / 512 / host-derived)	operator's number	operator reading the doc and setting the env. LOW-MEDIUM depending on value vs host RAM + pgx pool size; calibrate via the saturation benchmark.

4. Single most important tradeoff / risk (per item)

- **Item O:** the one header that can actually break the product is **Content-Security-Policy on the SPA document** — a too-strict script-src/ style-src can white-screen the manager UI. The mitigation is to keep Tier C (SPA CSP) behind a mandatory validation step (§6) and ship the SPA-safe Tier A/B first. Cross-Origin-Embedder-Policy: require-corp is the runner-up risk and is therefore **omitted** from the proposal outright.
- **Item Q:** the one risk is the **polarity inversion** — flipping the default to 256 makes HELIX_MAX_INFLIGHT=0 an *explicit* opt-out for unlimited. This is a behaviour change for any existing deployment relying on the current unlimited default (§11.4.122 territory), so it needs a documented .env.example + deployment-guide note and operator sign-off, not a silent flip.

5. If approved — validation plan (NOT executed here)

Any wiring that follows this brief MUST (per §11.4.108 / §11.4.115 / §11.4.169): 1. **Item O, Tier A/B:** httptest assertions that every response (incl. 401/429/500 error paths and the 429 shed) carries the Tier-A headers, and every API JSON response carries the Tier-B CSP + Cache-Control: no-store; a HSTS test proving it is present with TLS-config and ABSENT without. 2. **Item O, Tier C (SPA CSP):** load the *real built* manager SPA (host-rendered, §11.4.170) with the proposed CSP and assert zero CSP violations in the console + the UI renders (no white-

screen) — the §1.3.3 watch-items resolved empirically, not guessed. This is the gate that must pass BEFORE the SPA CSP is enabled. 3. **Item Q:** a saturation/DDoS test (the CONTINUATION “security saturation / DDoS” item) driving > limit concurrent requests against the live server, asserting 429 + Retry-After on the overflow and 200 on the in-budget set, capturing the host memory ceiling to confirm 256 is safe for the target — calibrate the shipped default against that captured evidence. 4. A paired §1.1 mutation for each new gate (strip a header / flip the default back) proving the gate FAILs.

Sources verified

- OWASP Secure Headers Project — “headers to add” recipe (machine-readable), ci/headers_add.json, last_update_utc: 2026-06-30 — https://raw.githubusercontent.com/OWASP/www-project-secure-headers/master/ci/headers_add.json (verified 2026-07-09)
- OWASP Secure Headers Project — “headers to remove” recipe, ci/headers_remove.json, last_update_utc: 2026-06-30 — https://raw.githubusercontent.com/OWASP/www-project-secure-headers/master/ci/headers_remove.json (verified 2026-07-09)
- OWASP Secure Headers Project landing page — <https://owasp.org/www-project-secure-headers/> (verified 2026-07-09)
- Go/Gin concurrency-limiter patterns (semaphore + shed overflow; resource-derived sizing): “Protect Your Go API with a Concurrency Limiter” — <https://medium.com/@dan.my1313/protect-your-go-api-with-a-concurrency-limiter-17502038e1b9> ; aviddiviner/gin-limit — <https://github.com/aviddiviner/gin-limit> ; gin issue #955 “how can I limit the number of current requests?” — <https://github.com/gin-gonic/gin/issues/955> (all verified 2026-07-09)
- Internal current-state evidence (working tree, verified 2026-07-09): server/internal/api/server.go:117, server/internal/api/middleware.go:22-42, server/internal/api/rate_limit.go:17-34, server/internal/api/embed.go:36-37,51,68, server/internal/config/config.go:60-62,126, server/internal/config/config.go:79-91; token benchmark harness server/internal/api/token_bench_test.go; measured timings (reject ≈473 ns / accept ≈1697 ns) operator-supplied.

This brief is a proposal only. No product code was modified. §11.4.6 honest boundary: the header set and default value proposed here are engineering recommendations backed by the cited OWASP recipe and the

measured benchmark — the final choices in §3 are the operator's, and the SPA CSP in particular is un-validated against the real built UI and MUST NOT ship until §5 step 2 passes.